THE MINISTRY OF SCIENCE AND HIGHER EDUCATION
OF THE RUSSIAN FEDERATION

ITMO University
(ITMO)

Faculty of Control Systems and Robotics

Task 3
for the subject
"Simulation of Robotic Systems"

Student:
Group No. R4137c *Mughal, Umer Ahmed Baig*
ITMO ID. 472268

Tutor:
Professor *Borisov, Ivan*

Assistant *Rakshin, Egor*

Saint Petersburg 2024

CONTENTS

0.1 Objective 1
0.2 Tasks 1

1 WAYS TO ACTUALIZE PASSIVE SYSTEMS WITH MOTORS IN XML .. 2

2 ADD ACTUATOR IN XML MODEL 3
2.1 Modifying XML Model by Adding Motor 3

3 PYTHON SCRIPT FOR EXECUTING THE XML MODEL 4
3.1 Defining Parameters 4
3.2 Computing Rotation Matrix and Euler Angles 4
3.3 Implementing PD Controller 4
3.4 Recording End-Effector Data 5
3.5 Running the Simulation 6
3.6 Visualizing Results 6

4 CONCLUSION 10

TASK DESCRIPTION

0.1 Objective:

Add a motor to the passive system at joint A or/and joint B according for the variant from the table. For example, if the actualization of motors A and B is given using sin wave, then both motors run on a sinusoid with the specified frequency and amplitude. The phase shift and sine wave shift are zero.

0.2 Tasks:

1. Explore ways to actualize the passive system using the motor in XML.
2. Modify the XML from the task 2.
3. Plot the position and orientation of the end-effector on graphs.

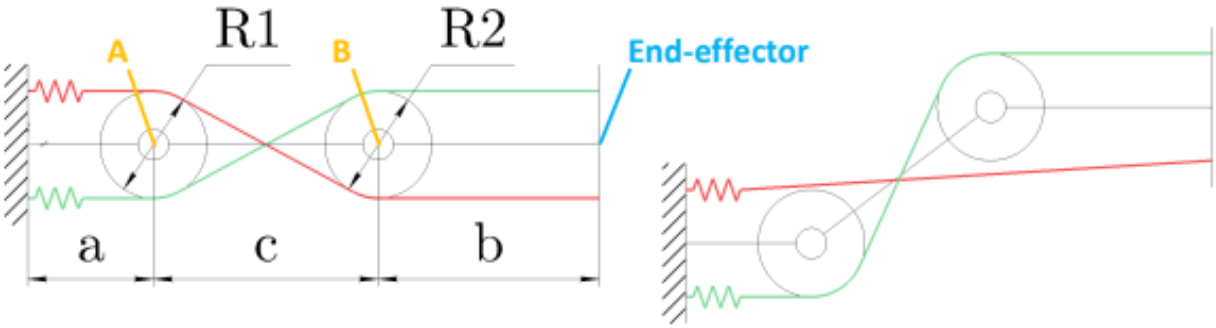

Figure 1: - XML Model with Actuators.

The initial data for completing the task are presented in Table 1.

Table 1

Motors	Amplitude	Frequency	Constant Angle
B	1.073 rad	3.2 Hz	1.564 rad

Given values of the parameters according to the task.

1 WAYS TO ACTUALIZE PASSIVE SYSTEMS WITH MOTORS IN XML

In robotic systems, motors are fundamental components that actively drive motion, control torque, and provide precision to the system. In MuJoCo XML models, motors can be customized to perform a variety of tasks, ranging from simple actuation to more complex behaviors like torque modulation. Below is a brief explanation of how motors can be utilized and configured to actualize a system in XML:

Torque-Based Actuation

Motors can be set to operate in torque mode, where they directly apply a torque to the joint. This is particularly useful for simulating systems where force control is required, such as robotic arms or compliant systems.

```
<actuator>
    <motor name="torque_motor" joint="joint_1" gear="1" />
</actuator>
```

Velocity Control

In systems requiring constant speed or dynamic velocity adjustments, motors can be configured for velocity control. This is commonly used in applications such as conveyor belts or rotational systems.

```
<actuator>
    <velocity name="velocity_motor" joint="joint_2" ctrllimited="true" ctrlrange="-5 5" />
</actuator>
```

Using Tendons with Motors

Tendons can be linked to motors to create distributed actuation. This approach is useful for systems where indirect motor control is needed, such as in pulley-based or cable-driven robots.

```
<actuator>
    <motor name="tendon_motor" tendon="tendon_1" />
</actuator>
<tendon>
    <spatial>
        <site site="start_site" />
        <site site="end_site" />
    </spatial>
</tendon>
```

The motor in XML models offers extensive configurability to actualize a variety of systems. By defining specific actuation methods, control parameters, and integrating feedback mechanisms, motors can simulate real-world behaviors effectively.

2 ADD ACTUATOR IN XML MODEL

2.1 Modifying XML Model by Adding Motor

Motors in MuJoCo are defined in the **<actuator>** section of the XML and are typically linked to joints to provide direct or indirect control over their motion. The **<position>** actuator type is specifically designed to control a joint's position by driving it toward a target value. Using the gear attribute, motors can scale the torque or force applied to the joint, which is critical for achieving precise actuation. The '**ctrllimited'** and '**ctrlrange**' attributes allow for constraining the range of motor control signals, preventing over-actuation and ensuring safe operation.

In the provided example, the actuator named "Motor B" is linked to the joint "B", enabling position control. The 'ctrllimited' attribute restricts the control signal within the specified 'ctrlrange' of -1 to 1, ensuring controlled and safe actuation. The gear attribute, set to 1, determines the scaling of the control signal.

To achieve the stated objective, the XML model from Task 2 is updated to include motor actuation. The code below was compiled to create an XML model with Actuator.

```
<actuator>
    <position name="Motor B" joint="B" ctrllimited="true" ctrlrange="-1 1" gear="1" />
</actuator>
```

Figure 2 illustrates the XML model with Motor B.

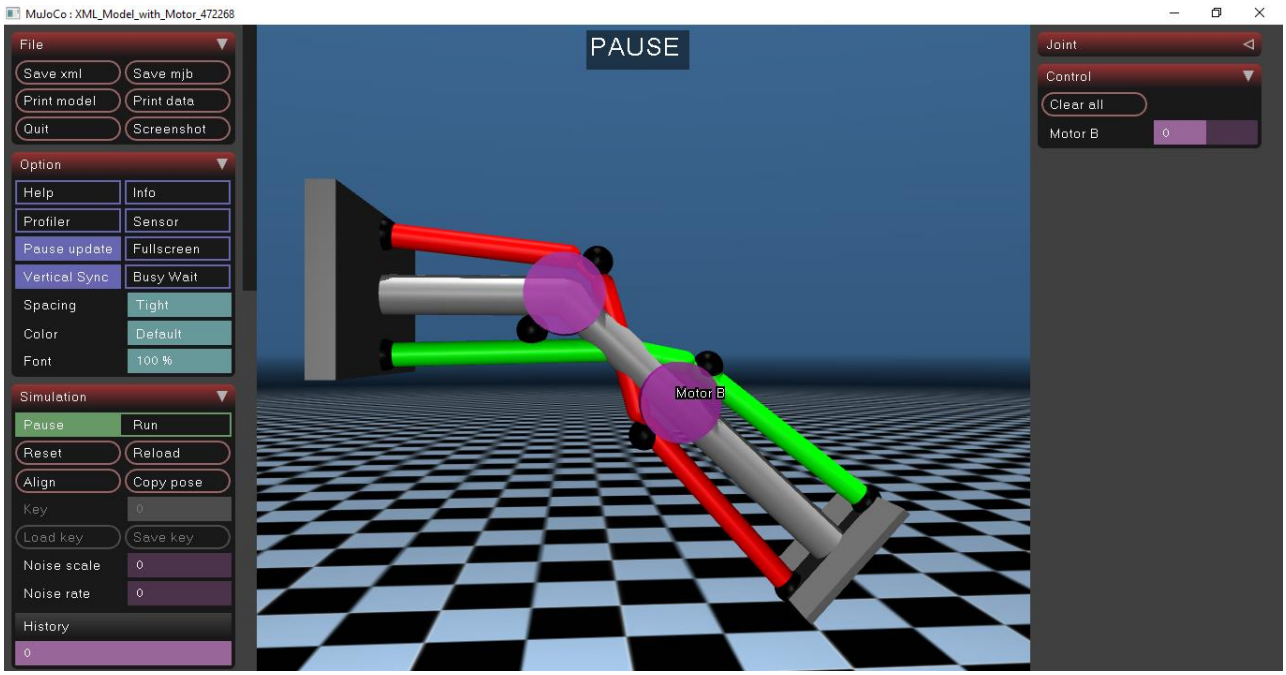

Figure 2: XML Model with Motor.

3 PYTHON SCRIPT FOR EXECUTING THE XML MODEL

Firstly, import the necessary libraries for simulation, visualization, numerical operations, and rotation computations. These libraries enable modeling, controlling, and analyzing of the robotic system. The MuJoCo XML model is loaded using the specified file path.

3.1 Defining Parameters

The below parameters define the target position for joint B. The amplitude and constant angle govern the range and frequency of the joint's oscillatory motion.

```
# Target position for joint B
amplitude = 1.073  # radians
frequency = 3.2  # Hz
constant_angle = 1.564  # radians
```

3.2 Computing Rotation Matrix and Euler Angles

The rotation matrix is extracted from the simulation state, representing the orientation of the geometry in 3D space. This matrix defines the relationship between the object's local coordinate system and the global coordinate system, encapsulating the object's rotational position. To make this information more interpretable, the rotation matrix is then converted into Euler angles, specifically pitch, roll, and yaw. These Euler angles provide a more human-readable representation of the geometry's orientation, with pitch representing the tilt around the lateral axis, roll around the forward axis, and yaw around the vertical axis. This transformation makes it easier to analyze and visualize the object's orientation during the simulation.

```
# Function to compute the rotation matrix and Euler angles (pitch, roll, yaw)
def get_rotation(model, data, site_name):
    wall_orientation = data.geom_xmat[mujoco.mj_name2id(model, mujoco.mjtObj.mjOBJ_GEOM, 'right_wall')]
    rotation_mat = np.reshape(wall_orientation, (3, 3))
    rotation = R.from_matrix(rotation_mat)
    euler_angles = rotation.as_euler('xyz', degrees=True)
    return euler_angles
```

3.3 Implementing the PD Controller

The PD (Proportional-Derivative) controller is a feedback control mechanism that adjusts the joint's actuation to achieve the desired position. The controller uses two key components: Proportional Gain (KP) and Derivative Gain (KD). The Proportional Gain (KP) is responsible for correcting the error between the target and current joint positions. It applies a force proportional to the magnitude of the position

error, driving the joint closer to the desired position. On the other hand, the Derivative Gain (KD) helps to dampen oscillations by considering the rate of change of the error, or the velocity of the joint. By incorporating the velocity of the joint, KD reduces overshooting and smoothing the movement, ensuring that the joint reaches its target more efficiently and without excessive oscillations. Together, these gains allow the PD controller to provide precise and stable control of the joint's motion.

```
# PD Controller function
def controller(model, data, joint_name, target_position):

    KP = 1  # Proportional gain
    KD = 0.05  # Derivative gain

    joint_id = mujoco.mj_name2id(model, mujoco.mjtObj.mjOBJ_JOINT, joint_name)
    joint_pos_addr = model.jnt_qposadr[joint_id]
    joint_vel_addr = model.jnt_dofadr[joint_id]

    current_position = data.qpos[joint_pos_addr]
    current_velocity = data.qvel[joint_vel_addr]
    position_error = target_position - current_position
    control_signal = KP * position_error - KD * current_velocity

    actuator_id = mujoco.mj_name2id(model, mujoco.mjtObj.mjOBJ_ACTUATOR, joint_name)
    data.ctrl[actuator_id] = control_signal
```

3.4 Recording End-Effector Data

The below code is responsible for recording the spatial position and pitch orientation of the end-effector, specifically the "right_wall" in the simulation. The position is captured by extracting the x, y, and z coordinates of the end-effector, which represent its motion in 3D space. In addition to the position, the function also records the orientation, specifically the pitch angle, which provides insight into the tilt or rotation of the end-effector relative to its horizontal plane. The pitch angle is a crucial metric for understanding the end-effector's tilt, which affects the overall motion and alignment. Together, the position and orientation data are key for evaluating the system's performance.

```
# Function to get the end-effector position and orientation
def get_end_effector_data():
    site_name = "right_wall"  # Name of the end-effector site
    site_id = mujoco.mj_name2id(m, mujoco.mjtObj.mjOBJ_GEOM, 'right_wall')
    end_effector_pos = d.geom_xpos[site_id]
    end_effector_positions["x"].append(end_effector_pos[0])
    end_effector_positions["y"].append(end_effector_pos[1])
    end_effector_positions["z"].append(end_effector_pos[2])
    euler_angles = get_rotation(m, d, site_name)
    end_effector_orientations_pitch.append(euler_angles[1])  # Pitch is the second element
```

3.5 Running the Simulation

The below code snippet sets up and executes the simulation loop. The PD controller updates joint B's position based on the target, while the simulation data is recorded. The viewer visualizes the system in real-time.

```python
# Launch viewer with simulation
with mujoco.viewer.launch_passive(m, d) as viewer:
    start_time = time.time()

    # Set the timestep to slow down the simulation
    timestep = 0.002  # 100 Hz (you can adjust this to make it slower)
    m.opt.timestep = timestep

    # Run the simulation for 10 seconds
    while viewer.is_running() and time.time() - start_time < 10:
        sim_time = time.time() - start_time
        target_position_joint_b = amplitude * np.sin(constant_angle * sim_time)
        target_positions.append(target_position_joint_b)
        controller(m, d, "B", target_position_joint_b)

        # Record joint positions
        joint_a_positions.append(d.qpos[m.jnt_qposadr[mujoco.mj_name2id(m, mujoco.mjtObj.mjOBJ_JOINT, 'A')]])
        joint_b_positions.append(d.qpos[m.jnt_qposadr[mujoco.mj_name2id(m, mujoco.mjtObj.mjOBJ_JOINT, 'B')]])

        # Record joint velocities
        joint_a_velocities.append(d.qvel[m.jnt_dofadr[mujoco.mj_name2id(m, mujoco.mjtObj.mjOBJ_JOINT, 'A')]])
        joint_b_velocities.append(d.qvel[m.jnt_dofadr[mujoco.mj_name2id(m, mujoco.mjtObj.mjOBJ_JOINT, 'B')]])

        # Record end-effector data
        get_end_effector_data()

        # Step the simulation
        mujoco.mj_step(m, d)

        # Synchronize the viewer
        viewer.sync()

        time.sleep(timestep)
```

3.6 Visualizing Results

The below code plot the simulation graphs to visualize the positions of joints A and B over time, showing their motion trajectories, illustrates the velocities of joints A and B, highlighting changes in speed over time. The 3D positional data of the end-effector is plotted to show how it moves in space over time.

```python
# Plot joint positions over time
plt.figure()
plt.plot(joint_a_positions, label="Joint A Position")
plt.plot(joint_b_positions, label="Joint B Position")
plt.xlabel("Time Steps")
plt.ylabel("Position (rad)")
plt.title("Joint Positions A & B Over Time")
plt.grid(True)
plt.legend()
plt.show()

# Plot joint velocities over time
plt.figure()
plt.plot(joint_a_velocities, label="Joint A Velocity")
plt.plot(joint_b_velocities, label="Joint B Velocity")
plt.xlabel("Time Steps")
```

```
plt.ylabel("Velocity (rad/s)")
plt.title("Joint Velocities A & B Over Time")
plt.grid(True)
plt.legend()
plt.show()

# Plot end-effector position
plt.figure()
plt.plot(end_effector_positions["x"], label="X Position")
plt.plot(end_effector_positions["y"], label="Y Position")
plt.plot(end_effector_positions["z"], label="Z Position")
plt.xlabel("Time Steps")
plt.ylabel("Position (m)")
plt.title("End-Effector Position (right_wall) Over Time")
plt.grid(True)
plt.legend()
plt.show()

# Plot end-effector orientation (pitch)
plt.figure()
plt.plot(end_effector_orientations_pitch, label="Pitch")
plt.xlabel("Time Steps")
plt.ylabel("Pitch (degrees)")
plt.title("End-Effector Pitch Orientation Over Time (right_wall)")
plt.grid(True)
plt.legend()
plt.show()
```

Throughout the simulation, the joint positions, velocities, and end-effector states are recorded, while a real-time viewer is continuously updated. After the simulation, the gathered data is plotted to evaluate the robot's motion and overall performance.

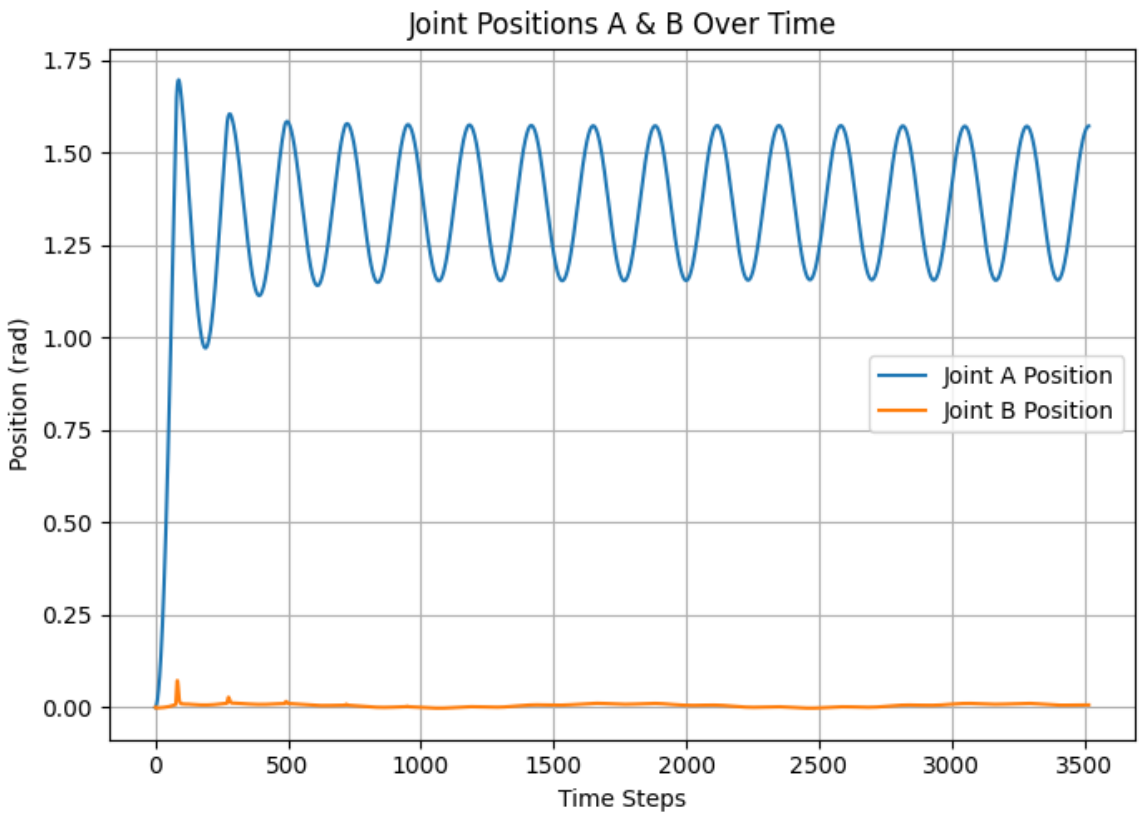

Figure 3: Joint Positions A & B over time.

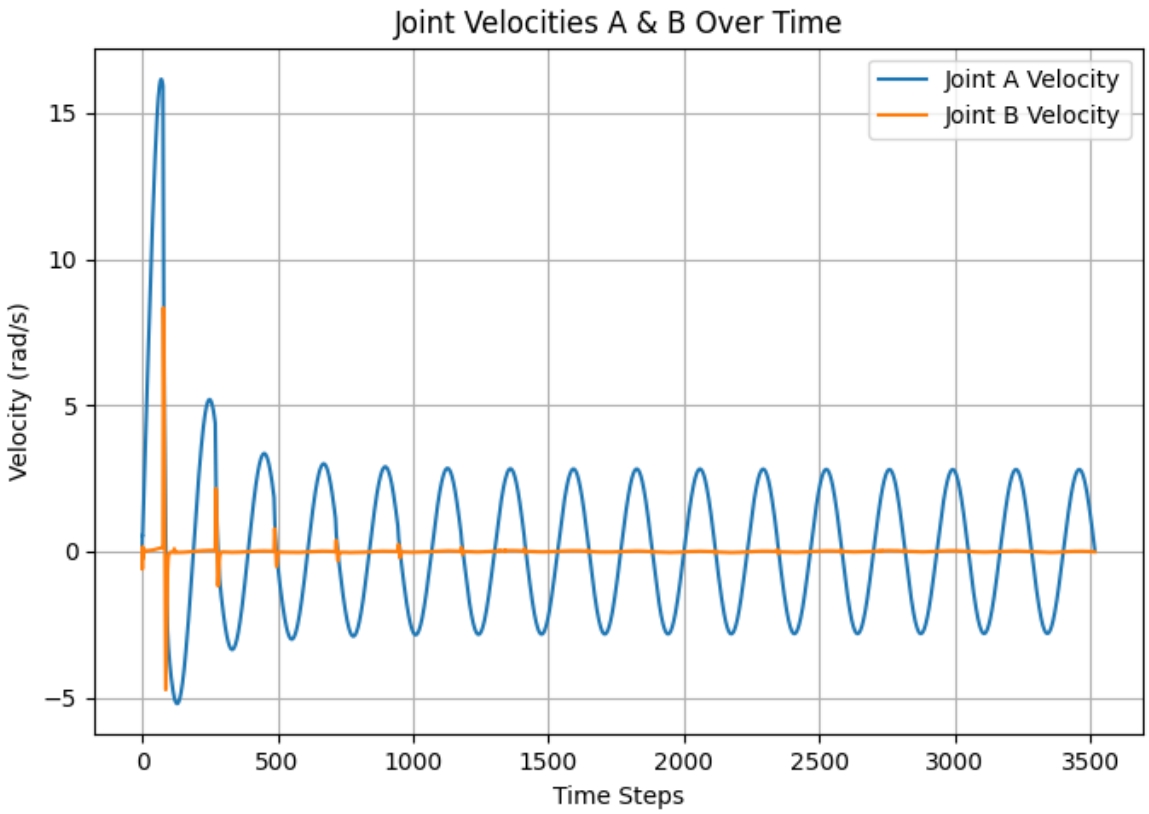

Figure 4: Joint Velocities A & B over time.

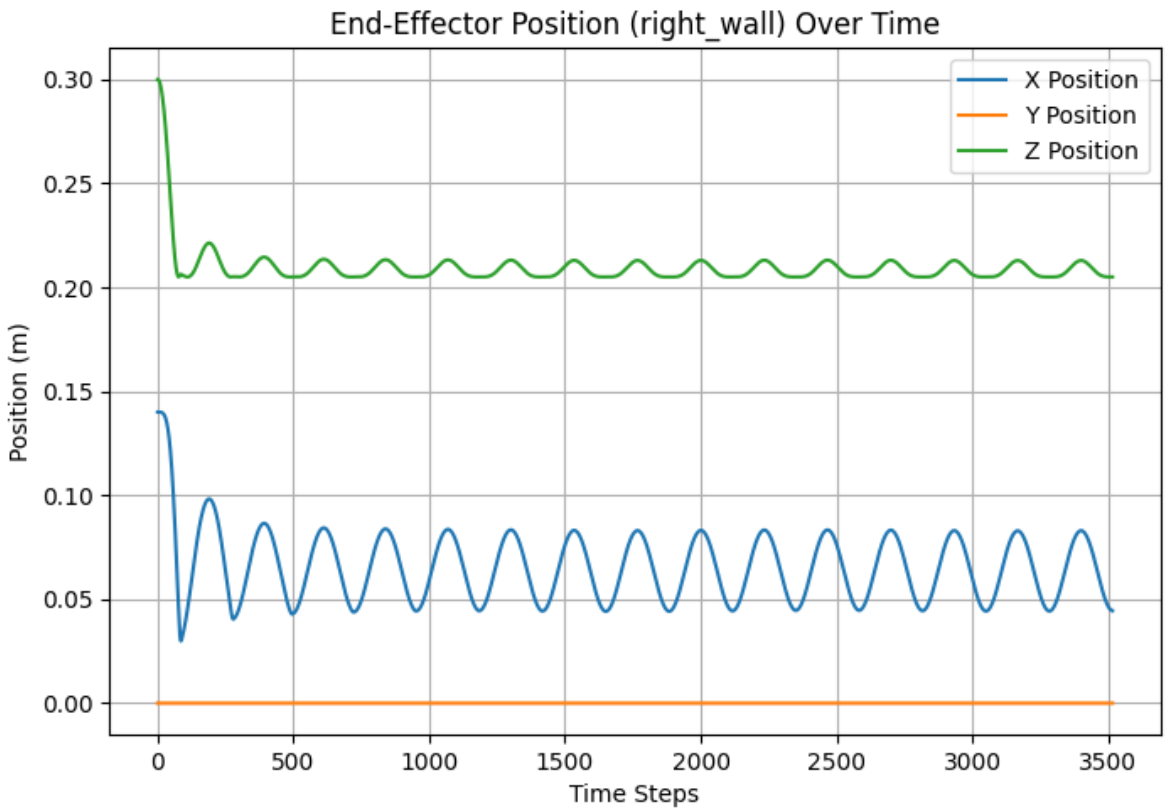

Figure 5: End-Effector Position (right_wall) over time.

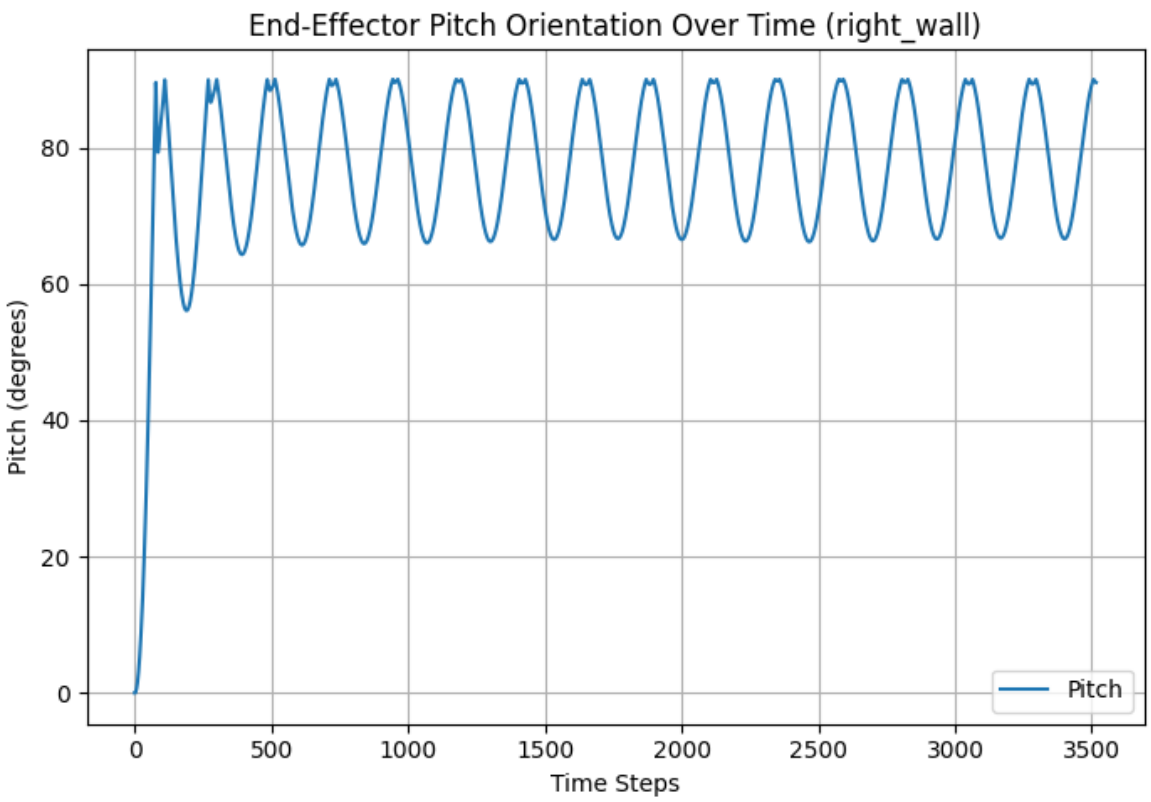

Figure 6: End-Effector (right_wall) Pith Orientation over time.

4 CONCLUSION

The simulation effectively demonstrates the dynamics and control of a robotic system using MuJoCo, with a focus on maintaining a constant angle trajectory for Joint B through a motorized actuator. By employing a PD controller, the system achieves stable and precise tracking of the target position, highlighting the effectiveness of feedback control in maintaining desired joint configurations. The logged data, including joint dynamics and end-effector behavior, provides valuable insights into the system performance under constant angle conditions. Through the integration of motor with position control algorithm (PD controller), the system ensures precise and efficient motion control while maintaining operational safety. This setup not only stabilizes joint motion but also optimizes the system's behavior for fixed trajectory profiles. The results showcase MuJoCo's capability to simulate and analyze robotic systems with high precision, enabling advanced control and optimization for robotic applications.